6

COLECCIONES Y ESTRUCTURAS DE DATOS

INTRODUCCIÓN

Son clases destinadas a aglutinar datos del mismo tipo o naturaleza. La cantidad de datos a aglutinar es indefinida. Todo ello en contraposición con clases definidas por el programador, con una finalidad encapsuladora de un conjunto de datos, concretados en atributos, en que el número de los mismos ya viene definido por los declarados en la propia clase. Además, los atributos de una clase pueden ser de diferente tipo.

Las clases a cuyo tratamiento se destina este tema, carecen de persistencia, es decir, se ubican en espacios de memoria que les destina la JVM. En ejemplo presentados en diferentes temas de este libro, se utilizan algunas de ellas, siendo dicha utilización integrada en pro de cumplir con los fines y objetivos de cada aplicación. En el tema destinado al tratamiento del polimorfismo, las similitudes y diferencias entre ellas, constituyen la base en que asentar los principios del polimorfismo, materializadas en las aplicaciones **PolimorfismoEstructurasDatos y PolimorfismoEstructurasDatos2**. En este tema, nos ceñiremos a presentar dichas clases, así como los mecanismos inherentes a su gestión, ilustrado todo ello mediante ejemplos muy sencillos y escuetos. El lector tendrá oportunidad en el resto de temas, en que como hemos mencionado, vuelven a aparecer integradas en aplicaciones de mayor envergadura, de reforzar los conceptos introducidos en este tema.

Tal y como hemos mencionado las clases “aglutinadoras” a tratar en este tema, aglutinan, valga la redundancia, datos primitivos, u objetos, todos ellos del mismo tipo. Así pues, la mayor parte de los ejemplos, consistirán en la gestión de un conjunto de objetos de la clase **Estado**, pero tratados con la idiosincrasia inherente a cada una de dichas clases “aglutinadoras”. Todo ello, en analogía con los ejemplos a que hemos hecho alusión presentados en el tema de este libro sobre el polimorfismo, que también utilizan objetos de la clase **Estado**, como elementos a aglutinar.

ARRAYS

Suponen una reminiscencia de la estructura de datos que ha venido estando presente en la mayoría de los lenguajes de programación desde los tiempos en que todavía no se había introducido el paradigma de la orientación a objetos, y se aplicaba únicamente la programación procedural. Son susceptibles de ser definidos con varias dimensiones, es decir, la posibilidad de utilizar varios índices para acceder a un componente en concreto integrado en el conjunto. A los arrays de una sola dimensión, se les ha llamado vectores, en determinados contextos, y a los de más de una dimensión, se les llama matrices. En este libro, vamos renunciar a aplicar dicha terminología, entre otras cosas, porque Java incorpora la clase *Vector*, cuya manipulación está totalmente encuadrada en la orientación a objetos, y a la que destinaremos un apartado de este tema. Con lo que con total independencia de las dimensiones con que sea declarado el array, uno o más de una, le llamaremos, simplemente, array.

Un array en Java, es un objeto, que aglutina un conjunto de componentes, todos del mismo tipo, que pueden ser tipos de datos primitivos, o referencias a objetos, y que, a diferencia del resto de clases tratadas en este libro, el número de componentes que tendrá, es fijo durante toda la ejecución del programa, definiéndose dicho número de componentes, en el momento de la instanciación del mismo. Exponemos a continuación una serie de ejemplos que ilustrarán al lector sobre la utilización de arrays.

```
package ejerciciosarrays1;
public class EjerciciosArrays1 {
public static void main(String[] args) {
// DECLARACION, INSTANCIACION, Y ASIGNACION DE VALORES
String[] meses =
{“Enero”,”Febrero”,”Marzo”,”Abril”,”Mayo”,”Junio”,”Julio”,”Agosto”,”Septiembre”,”Octubre”,”Noviembre”
// DECLARACION E INSTANCIACION DEL ARRAY
int[] lluvia = new int[12];
// int lluvia[] = new int[12];
// INTRODUCCION DE DATOS A SUS COMPONENTES
```

```
lluvia[0] = 23;
lluvia[1] = 15;
lluvia[2] = 34;
lluvia[3] = 51;
lluvia[4] = 68;
lluvia[5] = 20;
lluvia[6] = 10;
lluvia[7] = 15;
lluvia[8] = 120;
lluvia[9] = 180;
lluvia[10] = 90;
lluvia[11] = 68;
// int[] lluvia = {23, 15, 34, 51, 68, 20, 10, 15, 120, 180, 90, 68};
// VISUALIZACION DE DATOS ALMACENADOS EN LOS COMPONENTES DE LOS ARRAYS
for (int i=0; i<lluvia.length; i++)
System.out.println("la lluvia recogida en el mes "+meses[i]+" es : "+lluvia[i]+" l/m2");
// CALCULAR LLUVIA ACUMULADA ANUAL
int acumulaLluvia = 0;
for (int i=0; i<lluvia.length; i++)
acumulaLluvia += lluvia[i];
System.out.println("la pluviometría total anual es : "+acumulaLluvia+ " l/m2");
// OBTENER LA PLUVIOMETRIA MEDIA MENSUAL
float mediaMensualLluvia = ((float) acumulaLluvia) / lluvia.length;
System.out.println("la pluviometría media mensual es : "+String.format("%8.2f",mediaMensualLluvia)+ "
l/m2");
for (int lluviaMes : lluvia)
System.out.println(lluviaMes);
for (String nombreMes : meses)
System.out.println(nombreMes);
}
}
```

La ejecución de este programa visualiza en la consola de salida:

```
la lluvia recogida en el mes Enero es : 23 l/m2
la lluvia recogida en el mes Febrero es : 15 l/m2
la lluvia recogida en el mes Marzo es : 34 l/m2
la lluvia recogida en el mes Abril es : 51 l/m2
la lluvia recogida en el mes Mayo es : 68 l/m2
la lluvia recogida en el mes Junio es : 20 l/m2
la lluvia recogida en el mes Julio es : 10 l/m2
la lluvia recogida en el mes Agosto es : 15 l/m2
la lluvia recogida en el mes Septiembre es : 120 l/m2
iciembre };
la lluvia recogida en el mes Octubre es : 180 l/m2
la lluvia recogida en el mes Noviembre es : 90 l/m2
la lluvia recogida en el mes Diciembre es : 68 l/m2
la pluviometría total anual es : 694 l/m2
la pluviometría media mensual es : 57,83 l/m2
```

```
23
15
34
51
68
20
10
15
120
180
90
68
Enero
Febrero
Marzo
Abril
Mayo
Junio
Julio
Agosto
Septiembre
Octubre
Noviembre
Diciembre
```

Este programa utiliza dos arrays:

```
int[] lluvia
```

Es un array de componentes tipo de dato primitivo int, destinado cada uno de dichos componentes a almacenar la pluviometría total de cada uno de los 12 meses del año, en un lugar determinado.

```
String[] meses
```

Se trata de un array cuyos componentes son datos referenciales a objetos de la clase *String*, cuya finalidad es almacenar el nombre de los 12 meses del año.

Como si de cualquier otro objeto se tratase para utilizar un array, son necesarios una serie de pasos previos:

1. Declarar la referencia al objeto con que poder manipularlo posteriormente:

   ```
   int[] lluvia
   ```

   La declaración de una referencia a array también se puede implementar:

   ```
   int lluvia[]
   ```

2. Instanciar el objeto, asignando la referencia devuelta por el operador new, a la referencia previamente declarada a tal efecto:

```
lluvia = new int[12]
```

estableciendo entre "[ ]" el número de componentes que tendrá el array. El número de componentes del array, no podrá variar durante todo el ámbito de vida del mismo.

En el ejemplo hemos aplicado una declaración de referencia, e instanciación del objeto, abreviados:

```
int[] lluvia = new int[12];
```

Como reminiscencia de los lenguajes procedurales, para el acceso a los componentes del array, sigue utilizándose el operador indexación "[ ]", al que es necesario establecer el número de posición del componente dentro del conjunto: "[posición]". El número de posición en que se ubican los componentes, empieza a contar a partir del "0". Así pues, en el ejemplo almacenamos un dato en cada componente del array:

```
lluvia[0] = 23;
lluvia[1] = 15;
. . .
lluvia[11] = 68;
```

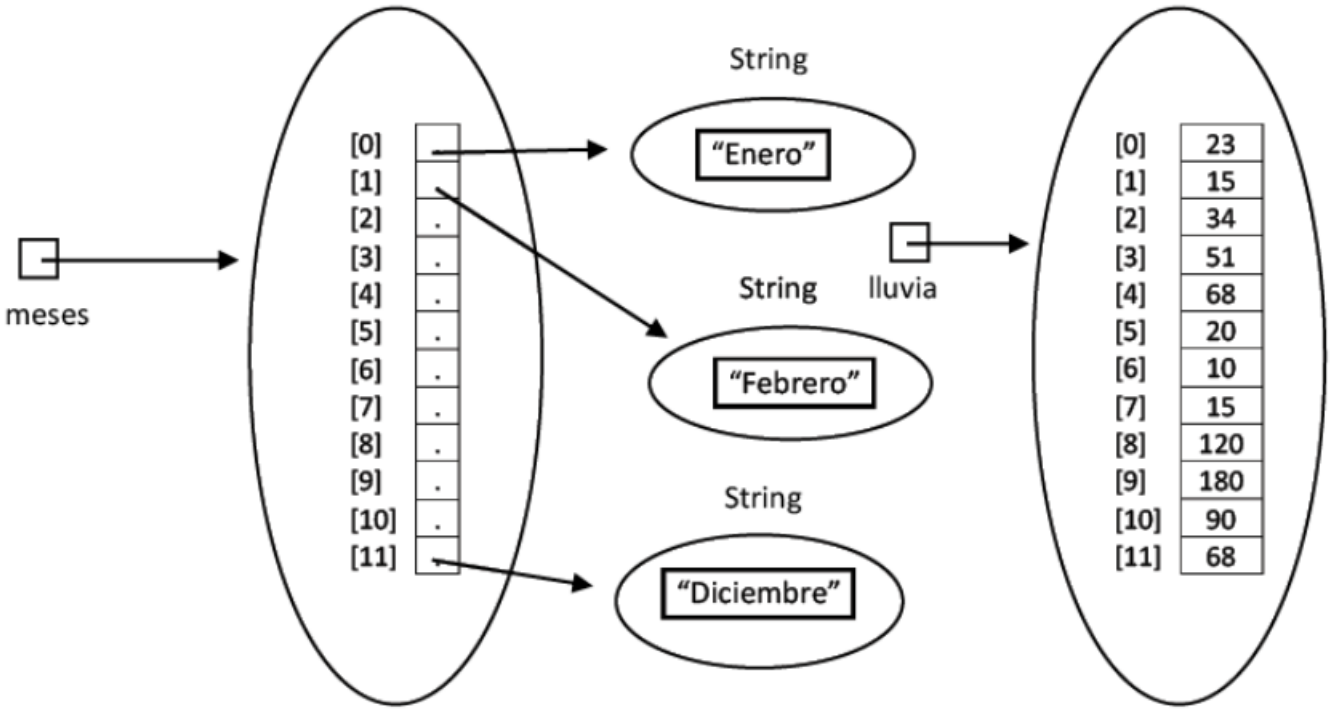
También podríamos haber procedido a simultanear la declaración abreviada, y la asignación de componentes, definiéndolos por enumeración entre "{ }", y separando por "," los diferentes valores:

```
int[] lluvia = {23, 15, 34, 51, 68, 20, 10, 15, 120, 180, 90, 68};
```

En el caso del array destinado a almacenar los nombres de los meses del año, hemos recurrido a esta variante de declaración, instanciación, y asignación de valores a los componentes, abreviada:

```
String[] meses =
{"Enero","Febrero","Marzo","Abril","Mayo","Junio","Julio","Agosto","Septiembre","Octubre","Noviembre"
```

Mediante el siguiente esquema dispondremos de una representación gráfica de ambos arrays, así como de los valores que se registran en los mismos.



En el ejemplo, se aplican dos "recorridos" a los componentes de los arrays, uno para visualizar el dato almacenado en cada componente, accediendo a dicho componente, mediante las expresiones:

```
meses[i]
lluvia[i]
```

aplicando en ambos, como expresión de índice a aplicar al operador indexación "[ ]", el valor que va tomando la variable de control "*i*" de la estructura repetitiva.

El segundo recorrido, se aplica solamente a los componentes del array *int[] lluvia*, para acumular en la variable *int acumulaLluvia*, el sumatorio de los valores almacenados en todos los componentes del array.

iciembre"};

Se ha aplicado como condición de repetición en ambos recorridos:

```
i < lluvia.length
```

donde *length* es un atributo del array que devuelve el número de componentes del array. En los casos del ejemplo, dicho atributo devuelve 12. No se confunda el atributo *length* de los arrays, con el método *length()* de la clase *String*.

Aportamos al final del ejemplo una modalidad novedosa de utilización de la estructura *for*, aplicada al recorrido de estructuras de datos y colecciones. La hemos aplicado a recorrer el array

```
int[] lluvia
for (int lluviaMes : lluvia)
System.out.println(lluviaMes);
y a
String[] meses
for (String nombreMes : meses)
System.out.println(nombreMes);
```

que como observamos, responde al esquema:

for (tipo_dato_del_componente dato_a_utilizar_en_bloque : estructura_o_colección)

Expondremos a continuación otro ejemplo, basado en la misma gestión, pero para tres ciudades diferentes. Un array de dos dimensiones nos permitirá dicha ampliación en el número de lugares, de los cuales pretendemos contemplar la pluviometría.

```
package ejerciciosarrays2;
public class EjerciciosArrays2 {
public static void main(String[] args) {
// DECLARACION, INSTANCIACION, Y ASIGNACION DE VALORES
String[] meses =
{“Enero”,”Febrero”,”Marzo”,”Abril”,”Mayo”,”Junio”,”Julio”,”Agosto”,”Septiembre”,”Octubre”,”Noviembre”,
String[] ciudades = {“Madrid”,”Sevilla”,”Valencia”};
// DECLARACION E INSTANCIACION DEL ARRAY
int[][] lluvia = new int[3][12];
int acumulaLluvia[] = new int[3];
float mediaMensualLluvia[] = new float[3];
// INTRODUCCION DE DATOS A SUS COMPONENTES
lluvia[0][0] = 23;
lluvia[0][1] = 15;
lluvia[0][2] = 34;
lluvia[0][3] = 51;
lluvia[0][4] = 68;
lluvia[0][5] = 20;
lluvia[0][6] = 10;
lluvia[0][7] = 15;
lluvia[0][8] = 120;
lluvia[0][9] = 180;
lluvia[0][10] = 90;
lluvia[0][11] = 68;
lluvia[1][0] = 18;
lluvia[1][1] = 12;
lluvia[1][2] = 30;
```

Descargado en: www.detodoprogramacion.org

```
  lluvia[1][3] = 51;
  lluvia[1][4] = 43;
  lluvia[1][5] = 22;
  lluvia[1][6] = 8;
  lluvia[1][7] = 15;
  lluvia[1][8] = 115;
  lluvia[1][9] = 175;
  lluvia[1][10] = 93;
  lluvia[1][11] = 72;
  lluvia[2][0] = 24;
  lluvia[2][1] = 16;
  lluvia[2][2] = 15;
  lluvia[2][3] = 50;
  lluvia[2][4] = 31;
  lluvia[2][5] = 14;
  lluvia[2][6] = 9;
  lluvia[2][7] = 19;
  lluvia[2][8] = 120;
  lluvia[2][9] = 130;
  lluvia[2][10] = 73;
  lluvia[2][11] = 58;
  // VISUALIZACION DE DATOS ALMACENADOS EN LOS COMPONENTES DE LOS ARRAYS
  for (int i=0; i<=2; i++)
  { System.out.println("Ciudad : "+ciudades[i]);
  for (int k=0; k<lluvia[i].length; k++)
  System.out.println("la lluvia recogida en el mes "+meses[k]+" es : "+lluvia[i][k]+" l/m2");
iciembre"};
  }
  // CALCULO LLUVIA ACUMULADA ANUAL Y PLUVIOMETRIA MEDIA MENSUAL POR CIUDADES
  for (int i=0; i<=2; i++)
  {
  acumulaLluvia[i] = 0;
  for (int k=0; k<lluvia[i].length; k++)
  { acumulaLluvia[i] += lluvia[i][k];
  }
  mediaMensualLluvia[i] = ((float) acumulaLluvia[i]) / lluvia[i].length;
  }
  // VISUALIZACION PLUVIOMETRIA ANUAL Y PLUVIOMETRIA MEDIA MENSUAL POR CIUDADES
  for (int i=0; i<=2; i++)
  { System.out.println("Ciudad : "+ciudades[i]);
  System.out.println("la pluviometría total anual es : "+acumulaLluvia[i]+ " l/m2");
  System.out.println("la pluviometría media mensual es :
  "+String.format("%8.2f",mediaMensualLluvia[i])+ " l/m2");
  }
  }
  }
```

La ejecución del programa, visualiza en la consola de salida:

```
Ciudad : Madrid
la lluvia recogida en el mes Enero es : 23 l/m2
la lluvia recogida en el mes Febrero es : 15 l/m2
la lluvia recogida en el mes Marzo es : 34 l/m2
la lluvia recogida en el mes Abril es : 51 l/m2
la lluvia recogida en el mes Mayo es : 68 l/m2
la lluvia recogida en el mes Junio es : 20 l/m2
la lluvia recogida en el mes Julio es : 10 l/m2
la lluvia recogida en el mes Agosto es : 15 l/m2
la lluvia recogida en el mes Septiembre es : 120 l/m2
la lluvia recogida en el mes Octubre es : 180 l/m2
la lluvia recogida en el mes Noviembre es : 90 l/m2
la lluvia recogida en el mes Diciembre es : 68 l/m2
Ciudad : Sevilla
la lluvia recogida en el mes Enero es : 18 l/m2
la lluvia recogida en el mes Febrero es : 12 l/m2
la lluvia recogida en el mes Marzo es : 30 l/m2
la lluvia recogida en el mes Abril es : 51 l/m2
la lluvia recogida en el mes Mayo es : 43 l/m2
la lluvia recogida en el mes Junio es : 22 l/m2
la lluvia recogida en el mes Julio es : 8 l/m2
la lluvia recogida en el mes Agosto es : 15 l/m2
la lluvia recogida en el mes Septiembre es : 115 l/m2
la lluvia recogida en el mes Octubre es : 175 l/m2
la lluvia recogida en el mes Noviembre es : 93 l/m2
la lluvia recogida en el mes Diciembre es : 72 l/m2
Ciudad : Valencia
la lluvia recogida en el mes Enero es : 24 l/m2
la lluvia recogida en el mes Febrero es : 16 l/m2
la lluvia recogida en el mes Marzo es : 15 l/m2
la lluvia recogida en el mes Abril es : 50 l/m2
la lluvia recogida en el mes Mayo es : 31 l/m2
la lluvia recogida en el mes Junio es : 14 l/m2
la lluvia recogida en el mes Julio es : 9 l/m2
la lluvia recogida en el mes Agosto es : 19 l/m2
la lluvia recogida en el mes Septiembre es : 120 l/m2
la lluvia recogida en el mes Octubre es : 130 l/m2
la lluvia recogida en el mes Noviembre es : 73 l/m2
la lluvia recogida en el mes Diciembre es : 58 l/m2
Ciudad : Madrid
la pluviometria total anual es : 694 l/m2
la pluviometria media mensual es : 57,83 l/m2
Ciudad : Sevilla
la pluviometria total anual es : 654 l/m2
la pluviometria media mensual es : 54,50 l/m2
Ciudad : Valencia
la pluviometria total anual es : 559 l/m2
la pluviometria media mensual es : 46,58 l/m2
```

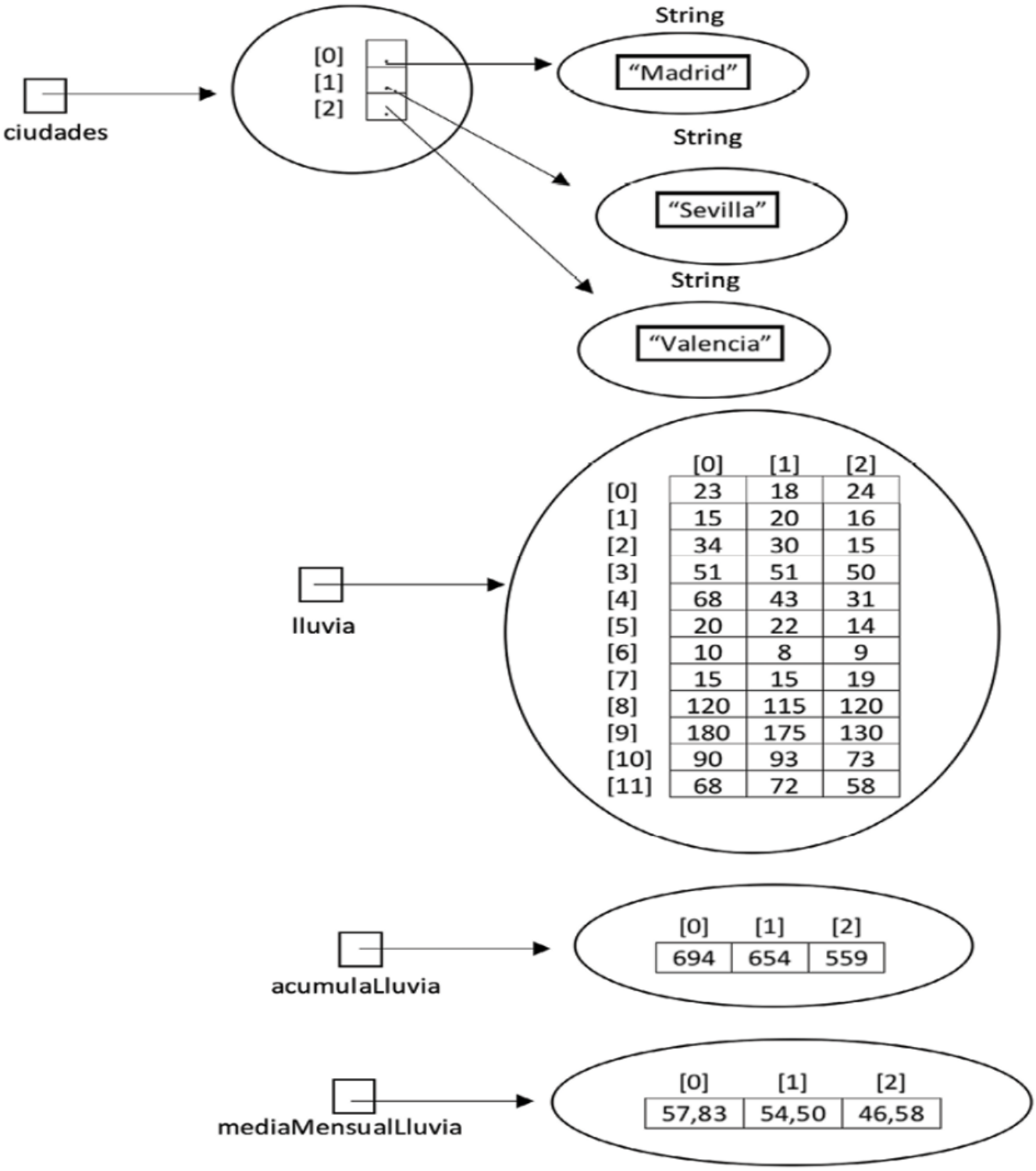
Hemos incorporado un nuevo array al ejemplo:

```
String[] ciudades = {“Madrid”,”Sevilla”,”Valencia”};
```

que registre los nombres de las ciudades a contemplar. Como ya no contemplamos un solo dato de pluviometría acumulada anual, y de media pluviometría, sino tantos como ciudades contemplemos, utilizaremos sendos arrays a tal efecto:

```
int acumulaLluvia[]
float mediaMensualLluvia[]
```

Representamos mediante el siguiente esquema, los arrays que intervienen, así como los valores que se registran en los mismos.


ciudades
[0]
[1]
[2]
String
"Madrid"
String
"Sevilla"
String
"Valencia"
lluvia
[0] [1] [2]
[0] 23 18 24
[1] 15 20 16
[2] 34 30 15
[3] 51 51 50
[4] 68 43 31
[5] 20 22 14
[6] 10 8 9
[7] 15 15 19
[8] 120 115 120
[9] 180 175 130
[10] 90 93 73
[11] 68 72 58
acumulaLluvia
[0] [1] [2]
694 654 559
mediaMensualLluvia
[0] [1] [2]
57,83 54,50 46,58

La existencia de los índices es lo que permite un acceso directo al componente que se pretende en cada momento, aplicando en el primer índice, el valor de ciudad que corresponda, y en el segundo índice, el valor del mes.

Obsérvese la utilización que se hace en este ejemplo del atributo *lengtht*:

```
lluvia[i].length
```

devolviéndonos el número de componentes para un valor concreto aplicado en primera dimensión.

- Un array puede tener hasta un máximo de 255 dimensiones, es decir, aplicar hasta un máximo de 255 expresiones de índice mediante operador indexación "[ ]", aplicados a la organización de los datos que se pretenden almacenar. En este ejemplo se utilizan las siguientes dimensiones, cada una de ellas destinada a organizar por:ciudades
- meses

lo cual nos ha permitido obtener estadísticas por ciudades

Es susceptible de ser ampliado, utilizando más dimensiones para organizar por:

- comunidades autónomas
- años

De esta forma podríamos ampliar el repertorio de estadísticas a aplicar a los datos registrados en el array.

Planteamos a continuación un nuevo ejemplo sobre arrays, en que los componentes del mismo son referencias a objetos encapsuladores de la información de un estado:

```
package ejerciciosarrays3;
public class EjerciciosArrays3 {
public static void main(String[] args) {
// DECLARACION, INSTANCIACION DEL ARRAY, E INSTANCIACION DE OBJETOS
Estado[] estados = {new Estado(1, "España", 46570000), new Estado(2, "Portugal", 10310000), new
Estado(3, "Alemania", 82790000), new Estado(4, "Francia", 67120000), new Estado(5, "Italia",
60590000)};
/*
// DECLARACION E INSTANCIACION DEL ARRAY
Estado[] estados = new Estado[5];
// INSTANCIACION DE OBJETOS
estados[0] = new Estado(1, "España", 46570000);
```

```
estados[1] = new Estado(2, “Portugal”, 10310000);
estados[2] = new Estado(3, “Alemania”, 82790000);
estados[3] = new Estado(4, “Francia”, 67120000);
estados[4] = new Estado(5, “Italia”, 60590000);
*/
for (int i=0; i<estados.length; i++)
{
System.out.println(“identificador de estado : “+estados[i].getIdEstado());
System.out.println(“nombre del estado : “+estados[i].getNombre());
System.out.println(“número de habitantes del estado : “+estados[i].getNumeroHabitantes());
System.out.println(“-----------------------------------------------”);
}
// BUSQUEDA DE LOS ESTADOS CON MAYOR Y MENOR NUMERO DE HABITANTES
int posicionEstadoMasPoblado = 0;
int poblacionMaxima = estados[0].getNumeroHabitantes();
int posicionEstadoMenosPoblado = 0;
int poblacionMinima = estados[0].getNumeroHabitantes();
for (int i=1; i<estados.length; i++)
{
if (estados[i].getNumeroHabitantes() > poblacionMaxima)
{
posicionEstadoMasPoblado = i;
poblacionMaxima = estados[i].getNumeroHabitantes();
}
else
if (estados[i].getNumeroHabitantes() < poblacionMinima)
{
posicionEstadoMenosPoblado = i;
poblacionMinima = estados[i].getNumeroHabitantes();
}
}
System.out.println(“El estado de mayor población es “+estados[posicionEstadoMasPoblado].getNombre()+
“ que cuenta con “+estados[posicionEstadoMasPoblado].getNumeroHabitantes());
System.out.println(“El estado de menor población es
“+estados[posicionEstadoMenosPoblado].getNombre()+ “ que cuenta con
“+estados[posicionEstadoMenosPoblado].getNumeroHabitantes());
}
}
```

```
package ejerciciosarrays3;
public class Estado implements java.io.Serializable {
private int idEstado;
private String nombre;
private int numeroHabitantes;
public Estado() {
}
public Estado(int idEstado, String nombre, int numeroHabitantes) {
this.idEstado = idEstado;
```

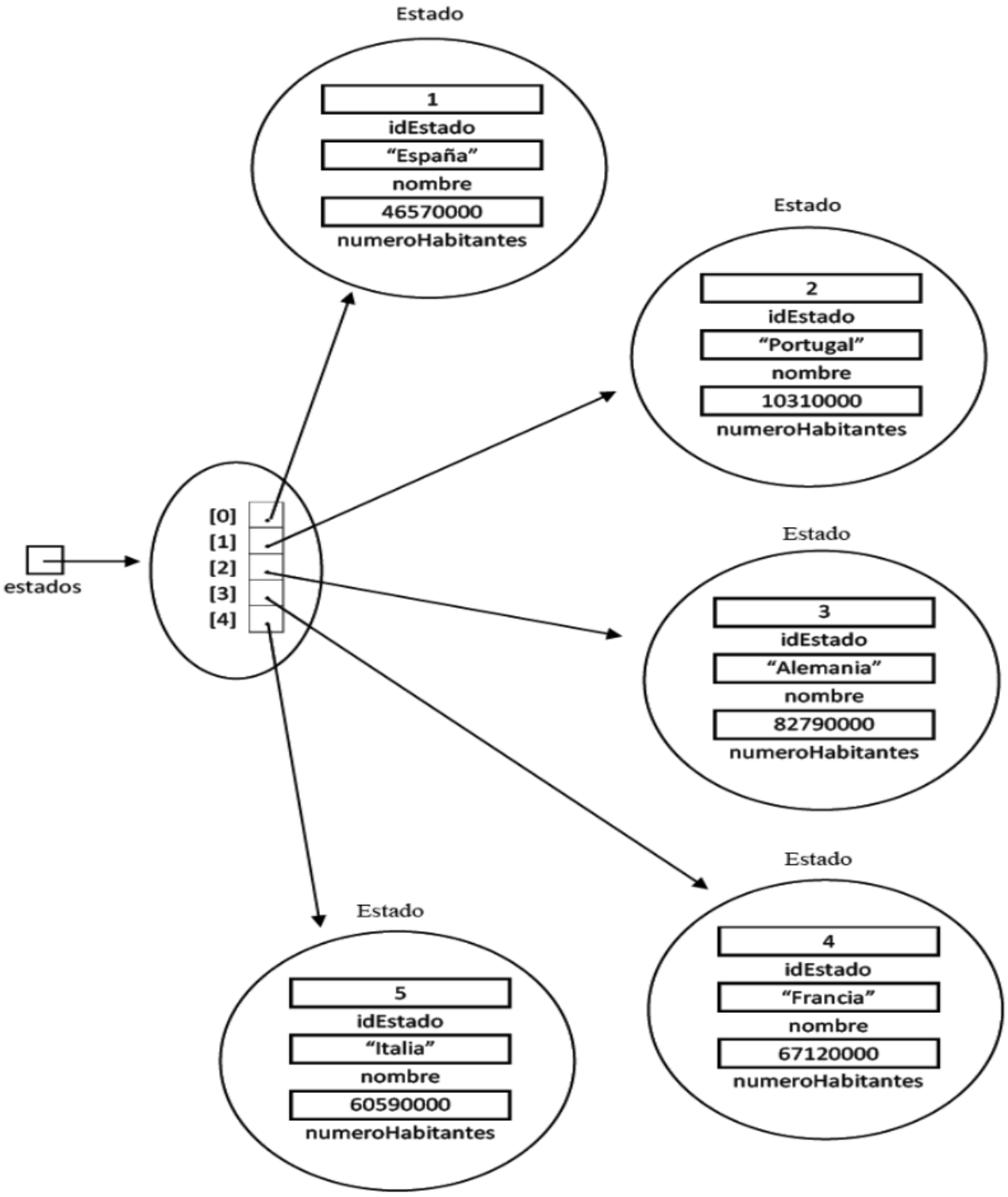
```
this.nombre = nombre;
this.numeroHabitantes = numeroHabitantes;
}
public int getIdEstado() {
return idEstado;
}
public void setIdEstado(int idEstado) {
this.idEstado = idEstado;
}
public String getNombre() {
return nombre;
}
public void setNombre(String nombre) {
this.nombre = nombre;
}
public int getNumeroHabitantes() {
return numeroHabitantes;
}
public void setNumeroHabitantes(int numeroHabitantes) {
this.numeroHabitantes = numeroHabitantes;
}
@Override
public boolean equals(Object objeto) {
boolean iguales = false;
if (objeto instanceof Estado) {
Estado estado = (Estado)objeto;
if ((this.idEstado == estado.idEstado) && (this.nombre.equals(estado.nombre) &&
(this.numeroHabitantes == estado.numeroHabitantes) ))
iguales = true;
}
return iguales;
}
@Override
public int hashCode() {
return idEstado + nombre.hashCode() + numeroHabitantes ;
}
}
```

La ejecución visualiza en la consola de salida:

```
identificador de estado : 1
nombre del estado : España
número de habitantes del estado : 46570000
-------------------------------------------
identificador de estado : 2
nombre del estado : Portugal
número de habitantes del estado : 10310000
-------------------------------------------
```

```
identificador de estado : 3
nombre del estado : Alemania
número de habitantes del estado : 82790000
-----------------------------------------
identificador de estado : 4
nombre del estado : Francia
número de habitantes del estado : 67120000
-----------------------------------------
identificador de estado : 5
nombre del estado : Italia
número de habitantes del estado : 60590000
-----------------------------------------
El estado de mayor población es Alemania que cuenta con 82790000
El estado de menor población es Portugal que cuenta con 10310000
```

El siguiente esquema proporciona una representación gráfica del array y de los objetos aglutinados desde el mismo:


Estado
1
idEstado
"España"
nombre
46570000
numeroHabitantes
Estado
2
idEstado
"Portugal"
nombre
10310000
numeroHabitantes
estados
[0]
[1]
[2]
[3]
[4]
Estado
3
idEstado
"Alemania"
nombre
82790000
numeroHabitantes
Estado
4
idEstado
"Francia"
nombre
67120000
numeroHabitantes
Estado
5
idEstado
"Italia"
nombre
60590000
numeroHabitantes

Centremos nuestra atención en las dos posibles formas de instanciar objetos de la clase Estado, y aglutinarlos al array. Se ha aplicado una declaración, instanciación del array, instanciación de objetos, y aglutinamiento de los mismos al array, todo ello abreviado:

```
Estado[] estados = {new Estado(1, “España”, 46570000), new Estado(2, “Portugal”, 10310000), new
Estado(3, “Alemania”, 82790000), new Estado(4, “Francia”, 67120000), new Estado(5, “Italia”,
60590000)};
```

La referencia devuelta por los operadores new, se ha aplicado directamente a la lista enumerada de componentes del array:

```
{referencia, referencia, referencia, referencia, referencia}
```

Si la instanciación de objetos de la clase Estado, hubiese sido progresiva en la medida avanza la ejecución del programa, y en consecuencia, el aglutinamiento de las referencias que “apuntan” a dichos objetos, también hubiese sido progresiva, hubiésemos recurrido a la solución “desactivada”, por estar comentarizada:

```
estados[0] = new Estado(1, “España”, 46570000);
estados[1] = new Estado(2, “Portugal”, 10310000);
estados[2] = new Estado(3, “Alemania”, 82790000);
estados[3] = new Estado(4, “Francia”, 67120000);
estados[4] = new Estado(5, “Italia”, 60590000);
```

Obsérvese que el acceso a los objetos, y a la información encapsulada en sus atributos, se ha producido, exactamente igual que ha venido produciéndose hasta ahora en todos los ejemplos que se han expuesto en el libro:

```
referencia . invocación_a_método
```

La única diferencia, es que ahora la referencia es un componente del array al que se accede mediante el operador indexación:

```
estados[i].getNumeroHabitantes()
```

El siguiente ejemplo que presentamos a continuación, puede parecer contradictorio con lo que se ha comentado al principio del tema en relación a que un array aglutina componentes que deben ser todos del mismo tipo, o referencias a la misma clase de objeto. Expongamos en principio el ejemplo, y procederemos posteriormente a la aclaración de esta aparente contradicción.

```
package ejerciciosarrays4;
public class EjerciciosArrays4 {
```

```
public static void main(String[] args) {
// DECLARACION E INSTANCIACION DEL ARRAY
Object[] objetos = new Object[3];
// INSTANCIACION DE OBJETOS
objetos[0] = new Estado(1, “España”, 46570000);
objetos[1] = “Lo que el viento se llevó”;
objetos[2] = new Integer(500);
Estado estado = (Estado) objetos[0];
String cadena = (String) objetos[1];
Integer entero = (Integer) objetos[2];
System.out.println(“identificador de estado : “+estado.getIdEstado());
System.out.println(“nombre del estado : “+estado.getNombre());
System.out.println(“número de habitantes del estado : “+estado.getNumeroHabitantes());
System.out.println(“-----------------------------------------”);
System.out.println(cadena);
System.out.println(“-----------------------------------------”);
System.out.println(“el número encapsulado en el objeto Integer es : “+entero.intValue());
}
}
```

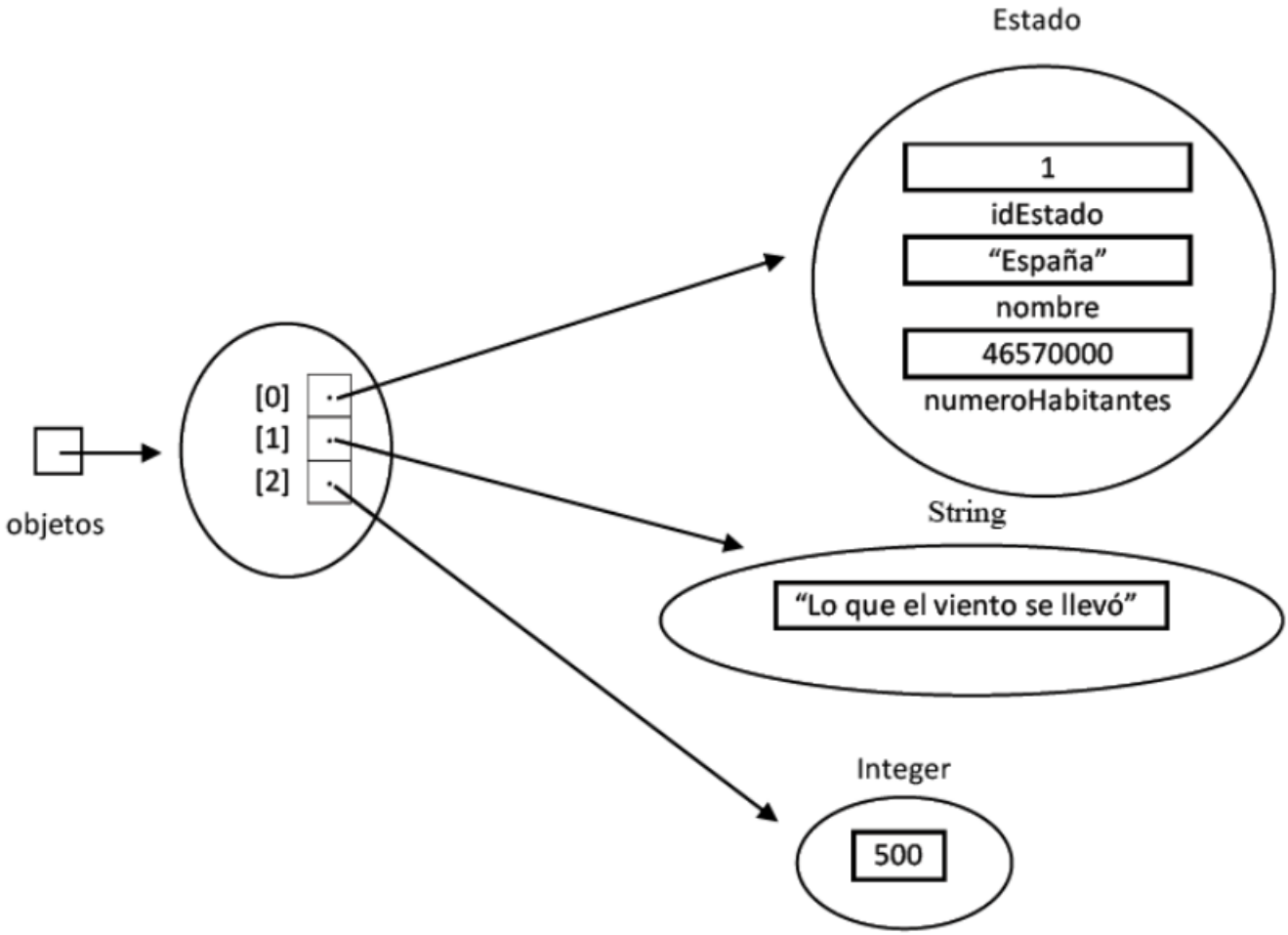
La ejecución visualiza en la consola de salida:

```
identificador de estado : 1
nombre del estado : España
número de habitantes del estado : 46570000
-----------------------------------------
Lo que el viento se llevó
-----------------------------------------
el número encapsulado en el objeto Integer es : 500
```

Se han instanciado tres objetos de clases diferentes:

- *Estado*, clase definida en el ejercicio anterior
- String
- Integer

El siguiente esquema supone una representación gráfica del array y de los objetos que aglutina, así como de los datos encapsulados:



Aclaremos la aparente contradicción. Es cierto que los objetos instanciados responden todos ellos a clases diferentes, pero por la declaración e instanciación del array:

```
Object[] objetos = new Object[3];
```

deducimos que está destinado a aglutinar un conjunto "homogéneo" de objetos, concretamente objetos que responden a la clase *Object*. Lo cual contrasta con lo "heterogéneo" de las clases a que responden los objetos aglutinados en el array. Es necesario tener en cuenta, como ya se ha comentado en el tema de este libro correspondiente a la herencia, que, en Java, todas las clases, tanto las predefinidas, como las definidas por el programador, tienen un ascendiente común en herencia, hablamos de la clase *Object*. En las asignaciones:

```
objetos[0] = new Estado(1, "España", 46570000);
```

```
objetos[1] = “Lo que el viento se llevó”;
objetos[2] = new Integer(500);
```

la referencia devuelta por el operador new, se asigna, en los tres casos a cada uno de los componentes del array, cuyo tipo base ha sido declarado como referencias a *Object*, con lo que nos encontramos con que referencias declaradas del tipo de la superclase, “apuntan” a objetos que responden al tipo de subclase. Esta posibilidad se trata en el tema correspondiente a polimorfismo. Cuando hacemos que un objeto sea “apuntado” por una referencia declarada a “apuntar” a su superclase, es como si al objeto “se le disfrazase” de superclase (valga la metáfora). En nuestro ejemplo, hemos “disfrazado” todos los objetos de una superclase común a todos ellos (*Object*), para poder aglutinarlos todos en el mismo array de referencias del tipo de dicha superclase común. Posteriormente, para poder acceder a métodos del objeto “apuntado” desde una referencia de un tipo de clase que no define dichos métodos (*Object* no define los métodos *getIdEstado(), getNombre(), getNumeroHabitantes(), intValue(),* etc.), es necesario “quitarles el disfraz” (seguimos con la metáfora), y que se muestren como realmente son, o “cambiarles el disfraz” a una superclase común que sí defina dichos métodos (no es este caso, pero sí el que se produce cuando implementamos polimorfismo). Ello se consigue, en el caso del ejemplo, aplicando cast a la subclase concreta a que responde el objeto:

```
Estado estado = (Estado) objetos[0];
String cadena = (String) objetos[1];
Integer entero = (Integer) objetos[2];
```

Este escenario que estamos presentando de aglutinar en un array de referencias a *Object*, objetos de cualquier clase, tanto predefinida, como definida por el programador, es decir, crear un “paquete” heterogéneo de objetos, es un recurso a utilizar muy valioso. En unos casos porque lo creemos conveniente por cuestiones de organización, por ejemplo, cuando nos planteamos transferir “en bloque”, un conjunto heterogéneo de objetos. Hay otros casos en nos vemos obligados a ello. Un ejemplo de obligatoriedad, se presenta, cuando se da la circunstancia de que un método debe devolver varios datos. Como ya se ha tratado en temas anteriores, un método, solamente puede devolver un único dato, acompañando a *return*. Ahora bien, dicho dato puede ser perfectamente, un “paquete”, del tipo a que hemos hecho alusión. Este recurso que estamos describiendo se utiliza en diferentes ejemplos de este libro.

BÚSQUEDAS: SECUENCIAL, Y BINARIA

Dedicamos este apartado, de forma monográfica a dos algoritmos. Ambos se implementarán en aplicar búsquedas en un array, dado que es la única estructura de datos tratada hasta el momento, pero son de aplicación general, sobre todo la búsqueda secuencial, en la mayor parte de estructuras de datos, colecciones, E/S en ficheros, XML, etc. De hecho, aparece en muchos ejemplos de este libro. Una búsqueda implica la localización de un dato en un conjunto de datos, si existe. La no existencia del dato a buscar en el conjunto, en su caso, ya es en sí información. Al mismo tiempo, también se pretende mediante estos ejercicios reforzar aspectos ya tratados con anterioridad, tales como las estructuras de control, y la clase *String*; sin obviar el disponer de un ejemplo más sobre arrays.

Exponemos, en primer lugar, el ejemplo de la búsqueda secuencial:

```
package busquedasecuencial;
public class BusquedaSecuencial {
public static void main(String[] args) {
String[] ciudades = {“Marsella”, “Roma”, “Montpellier”, “Sevilla”, “Oslo”, “Estocolmo”, “Venecia”,
“Carcassonne”, “Munich”, “Brest”};
String ciudadABuscar = “Estocolmo”; // nombre de ciudad existente en el array
// String ciudadABuscar = “Nápoles”; // nombre de ciudad no existente en el array
boolean ciudadEncontrada = false;
int posicionCiudadEncontrada= 0;
int contadorPosiciones = 0;
while ((contadorPosiciones < ciudades.length) && (!ciudadEncontrada)) {
if (ciudades[contadorPosiciones].compareTo(ciudadABuscar) == 0) {
posicionCiudadEncontrada = contadorPosiciones;
ciudadEncontrada = true;
}
contadorPosiciones ++;
}
if (ciudadEncontrada)
System.out.println(“Se encontró el nombre de ciudad : “+ciudades[posicionCiudadEncontrada]+” ubicada
en la posición : “+posicionCiudadEncontrada+” del array”);
else
System.out.println(“No se encontró el nombre de ciudad : “+ciudadABuscar);
}
}
```

La ejecución visualiza en la consola de salida:

```
Se encontró el nombre de ciudad : Estocolmo ubicada en la posición : 5 del array
```

El núcleo del algoritmo es una estructura repetitiva que recorre el conjunto de datos a buscar, y anidada a ella, se compara el elemento a buscar con cada uno de los tratados en cada repetición.

Utilizamos una variable que actúa como flag, inicializada a *false*, y que pasa a valer *true* cuando se localiza el dato buscado, si se localiza. Así pues, para cada repetición, se exige una doble condición, que no hayamos llegado al final del recorrido, y que todavía no se haya encontrado. Con lo que la búsqueda cesa cuando se ha llegado al final del recorrido sin localizar el elemento buscado, o cuando se ha localizado.

La búsqueda secuencial se aplica cuando los datos que integran el conjunto sobre el que se aplicará la búsqueda no responden a ningún tipo de orden, es decir, están desordenados. Recordemos las antiguas guías telefónicas, que todavía veíamos usar hasta hace pocos años, en domicilios particulares, empresas, y corporaciones de todo tipo. Eran usadas, cuando conociendo el nombre de una persona, empresa, o corporación, se pretendía localizar dicho nombre porque adjunto al mismo iba el número de teléfono asociado. Era impensable, buscar secuencialmente un nombre, en los cientos de páginas de letra apretada, de una guía telefónica. Todo el conjunto de miles, tal vez millones de nombres sobre el que se iba a realizar la búsqueda, estaban dispuestos según un criterio de ordenación alfabética. Dicha ordenación permitía, aplicando una búsqueda manual, la localización de un nombre en segundos. Ese procedimiento de búsqueda que se aplicaba sobre una guía telefónica, automática e inconscientemente, da lugar al algoritmo de búsqueda binaria, o dicotómica, que trataremos a continuación mediante el siguiente ejemplo:

```
package busquedabinaria;
public class BusquedaBinaria {
public static void main(String[] args) {
String[] ciudades = {“Brest”, “Carcassonne”, “Estocolmo”, “Marsella”, “Montpellier”, “Munich”,
“Oslo”, “Roma”, “Sevilla”, “Venecia”};
String ciudadABuscar = “Munich”; // nombre de ciudad existente en el array
// String ciudadABuscar = “Nápoles”; // nombre de ciudad no existente en el array
boolean ciudadEncontrada = false;
int limiteInferiorIntervalo = 0;
int limiteSuperiorIntervalo = ciudades.length - 1;
int posicionAComparar = (limiteInferiorIntervalo + limiteSuperiorIntervalo) / 2;
while ((limiteInferiorIntervalo <= limiteSuperiorIntervalo) && (!ciudadEncontrada)) {
visualizaValoresProcesoBusqueda(ciudades, ciudadABuscar, limiteInferiorIntervalo,
limiteSuperiorIntervalo, posicionAComparar);
if (ciudades[posicionAComparar].compareTo(ciudadABuscar) == 0) {
ciudadEncontrada = true;
}
else
{
if (ciudades[posicionAComparar].compareTo(ciudadABuscar) < 0)
{ limiteInferiorIntervalo = posicionAComparar + 1;
}
```

```
else
{ limiteSuperiorIntervalo = posicionAComparar - 1;
}
posicionAComparar = (limiteInferiorIntervalo + limiteSuperiorIntervalo) / 2;
}
}
if (ciudadEncontrada)
System.out.println("Se encontró el nombre de ciudad : "+ciudades[posicionAComparar]+" ubicada en la
posición : "+posicionAComparar+" del array");
else
System.out.println("No se encontró el nombre de ciudad : "+ciudadABuscar);
}
public static void visualizaValoresProcesoBusqueda(String[] ciudades, String ciudadABuscar, int
limiteInferiorIntervalo, int limiteSuperiorIntervalo, int posicionAComparar) {
System.out.println("límite inferir intervalo : "+limiteInferiorIntervalo);
System.out.println("límite superior intervalo : "+limiteSuperiorIntervalo);
System.out.println("posición a comparar : "+posicionAComparar);
int resultadoComparacion = ciudades[posicionAComparar].compareTo(ciudadABuscar);
System.out.println(resultadoComparacion);
System.out.println("----------------------------------------");
}
}
```

La ejecución del ejemplo visualiza en la consola de salida:

```
límite inferir intervalo : 0
límite superior intervalo : 9
posición a comparar : 4
-6
----------------------------------------
límite inferir intervalo : 5
límite superior intervalo : 9
posición a comparar : 7
5
----------------------------------------
límite inferir intervalo : 5
límite superior intervalo : 6
posición a comparar : 5
0
----------------------------------------
Se encontró el nombre de ciudad : Munich ubicada en la posición : 5 del array
```

Estructuralmente hablando, el algoritmo es análogo al anterior, una estructura repetitiva, a la que se anida una alternativa simple cuya condición determina si hemos localizado, o no, el elemento a buscar; así como la variable flag *boolean ciudadEncontrada*, que cumple un cometido totalmente análogo. Para mejor comprensión del algoritmo expuesto, aplicaremos una analogía entre los

valores que van tomando las variables utilizadas, con el procedimiento manual que se aplicaba en las búsquedas en las antiguas guías telefónicas. Asimismo, en cada repetición de la estructura repetitiva, se invoca al método *visualizaValoresProcesoBusqueda*, que va visualizando en la consola de salida, los valores que van tomando las variables utilizadas en el proceso. Recordemos como se efectuaba una búsqueda en las antiguas guías telefónicas. Se cogía la guía con las dos manos, e iniciábamos la búsqueda

- mano izquierda, aplicada al inicio de la guía, representada en el algoritmo por

  ```
  int limiteInferiorIntervalo
  ```

- mano derecha, aplicada al final de la guía, representada en el algoritmo por

  ```
  int limiteSuperiorIntervalo
  ```

- abríamos la guía por la mitad (en realidad, no era la mitad exacta, sino cierta aproximación ya tendenciosa hacia donde creíamos que podía estaba el dato), y en la página por la que abríamos la guía, mirábamos un dato de dicha página, representado en el algoritmo por

  ```
  ciudades[posicionAComparar]
  ```

Se comprobaba si el dato visualizado en la guía, era anterior, o posterior al que se buscaba, dada la ordenación alfabética. Análogamente a ello, utilizamos en el algoritmo el método *compareTo()* de la clase *String*, que compara la cadena encapsulada en el *String*, con la cadena que se pasa como argumento, devolviendo

- valor 0, si ambas cadenas son iguales
- valor negativo, si la primera cadena está posicionada antes, según el criterio de ordenación del dato, que la segunda
- valor positivo, en caso contrario

Si el dato buscado era posterior, la mano izquierda (*limiteInferiorIntervalo*) avanzaba posicionándose en la página siguiente a aquella en que habíamos fijado la atención, manteniéndose fija en el final la mano derecha (*limiteSuperiorIntervalo*). Caso de que el dato buscado fuese anterior, la que se mantenía fija era la mano izquierda (*limiteInferiorIntervalo*), y la mano derecha (*limiteSuperiorIntervalo*) se posicionaba sujetando la página anterior a la visualizada. Se volvía a repetir la búsqueda, abriendo otra vez por la página intermedia, pero siendo ahora el intervalo de páginas a buscar, la mitad anterior o posterior, según el resultado de la comparación. Y así

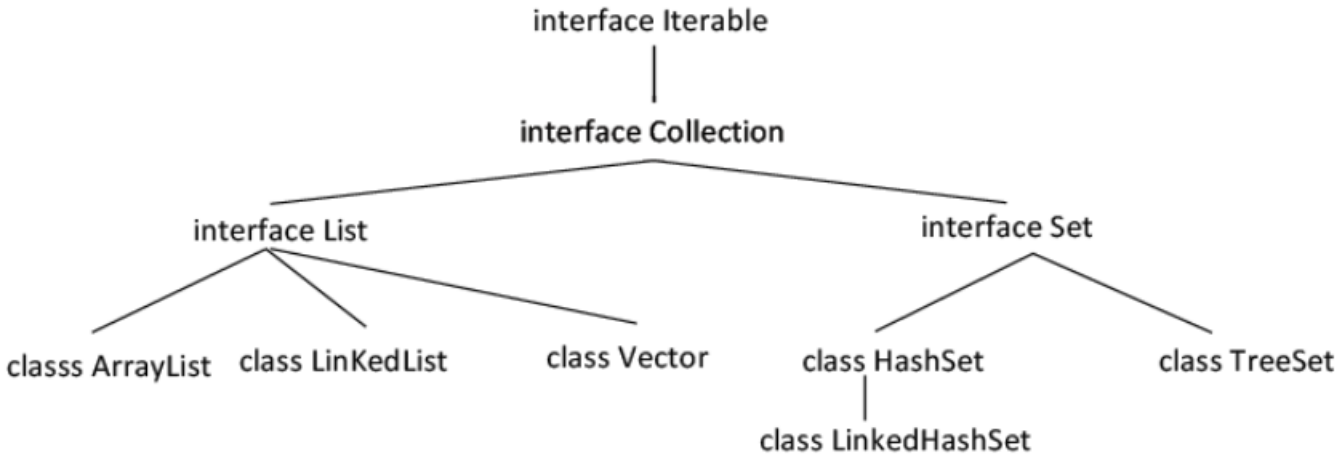
progresivamente íbamos acotando el rango de páginas a buscar, hasta que localizábamos el nombre buscado, si estaba en la página intermedia que habíamos abierto. O bien disminuyendo el rango progresivamente a mitades, quedaba como rango una sola página, y en ella no estaba el nombre sometido a búsqueda. En el algoritmo, la situación de no encontrarlo viene determinada cuando

```
limiteInferiorIntervalo > limiteSuperiorIntervalo
```

Obviamente, el rendimiento de la búsqueda binaria, es muy superior al de la secuencial, pero precisa que el conjunto de datos en el que se realizará la búsqueda esté previamente ordenado.

COLLECTION

Trataremos en este apartado la jerarquía de clases que permiten aglutinar un conjunto de datos, "encabezada" por la *interface Collection*. Presentamos dicha jerarquía en el siguiente esquema:



Procedemos a continuación a una breve descripción comparativa de todas las interfaces y clases de dicha jerarquía.

- ***Set*** **.-** No admite duplicados. No es sincronizado.
- ***List*** **.-** Admite duplicados. Aporta mejoras en relación a *Set*, entre ellas, permite un acceso directo a los elementos.
- ***Vector*****.-** Es sincronizado.
- ***ArrayList*** **.-** No es sincronizado.

- ***LinkedList .-*** Es preferible al *ArrayList* cuando se prevé mayor actividad de inserciones y eliminaciones, en relación a actividad de búsquedas.
- ***LinkedHashSet .-*** Los elementos quedan almacenados en el mismo orden en que se insertan.
- ***HashSet .-*** A diferencia de la anterior, el orden en que se recuperan los elementos, no coincide con el de inserción. Supone mayor rendimiento.
- ***TreeSet .-*** Los elementos se insertan según el criterio de ordenación.

También presentaremos en este apartado la *inteface Iterator*, y la *interface Enumeration*, que nos permiten aplicar un recorrido a las clases "descendientes" de *Collection*. Exponemos a continuación una serie de ejemplos de utilización representativos de la gestión de dichas clases. Dichos ejemplos "aglutinan" objetos de la clase **Estado**, ya expuesta y utilizada con anterioridad en este tema.

```
package ejerciciosarraylist;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Enumeration;
import java.util.Iterator;
public class EjerciciosArrayList {
public static void main(String[] args) {
Estado estado1;
// INSTANCIACION ArrayList
ArrayList<Estado> estados = new ArrayList();
// ArrayList estados = new ArrayList();
// INSTANCIACION DE OBJETOS Y AGLUTINADO AL ArrayList
estado1 = new Estado(1, "España", 46570000);
estados.add(estado1);
estado1 = new Estado(2, "Portugal", 10310000);
estados.add(estado1);
Estado estadoAEliminar = estado1; // retenemos el valor de la referencia para eliminarlo
posteriormente
estado1 = new Estado(3, "Alemania", 82790000);
estados.add(estado1);
estado1 = new Estado(4, "Francia", 67120000);
estados.add(estado1);
estado1 = new Estado(5, "Italia", 60590000);
estados.add(estado1);
System.out.println("----------------- RECORRIDO size() get() -----------------");
Estado estado2;
for (int i = 0; i<estados.size(); i++)
{
estado2 = estados.get(i);
// estado2 = (Estado)listaEstados.get(i);
```

```
System.out.println("identificador de estado : "+estado2.getIdEstado());
System.out.println("nombre del estado : "+estado2.getNombre());
System.out.println("número de habitantes del estado : "+estado2.getNumeroHabitantes());
System.out.println("-----------------------------------------------");
}
System.out.println("------ RECORRIDO for (Estado estado3 : listaEstados) -------");
for (Estado estado3 : estados)
{
System.out.println("identificador de estado : "+estado3.getIdEstado());
System.out.println("nombre del estado : "+estado3.getNombre());
System.out.println("número de habitantes del estado : "+estado3.getNumeroHabitantes());
System.out.println("-----------------------------------------------");
}
System.out.println("-------------------- RECORRIDO Iterator --------------------");
Estado estado4;
Iterator<Estado> iteratorEstados = estados.iterator();
while (iteratorEstados.hasNext()) {
estado4 = iteratorEstados.next();
System.out.println("identificador de estado : "+estado4.getIdEstado());
System.out.println("nombre del estado : "+estado4.getNombre());
System.out.println("número de habitantes del estado : "+estado4.getNumeroHabitantes());
System.out.println("-----------------------------------------------");
}
System.out.println("------------------ RECORRIDO Enumeration -------------------");
Estado estado5;
Enumeration<Estado> enumerationEstados = Collections.enumeration(estados);
while (enumerationEstados.hasMoreElements()) {
estado5 = enumerationEstados.nextElement();
System.out.println("identificador de estado : "+estado5.getIdEstado());
System.out.println("nombre del estado : "+estado5.getNombre());
System.out.println("número de habitantes del estado : "+estado5.getNumeroHabitantes());
System.out.println("-----------------------------------------------");
}
System.out.println("---------------- RECORRIDO desde el array ------------------");
Object[] arrayEstados = estados.toArray();
for (int i = 0; i<arrayEstados.length; i++)
{
System.out.println("identificador de estado : "+((Estado) arrayEstados[i]).getIdEstado());
System.out.println("nombre del estado : "+((Estado) arrayEstados[i]).getNombre());
System.out.println("número de habitantes del estado : "+((Estado)
arrayEstados[i]).getNumeroHabitantes());
System.out.println("-----------------------------------------------");
}
estados.remove(estadoAEliminar); // eliminamos el objeto que ocupa la posición en el ArrayList (el
cuarto)
System.out.println("Después de haber eliminado un estado, quedan en el ArrayList "+estados.size()+"
objetos");
estados.clear();
```

```
System.out.println("Después de haber aplicado el método clear() quedan en el ArrayList
"+estados.size()+" objetos");
}
}
```

```
package ejerciciosvector;
import java.util.Enumeration;
import java.util.Iterator;
import java.util.Vector;
public class EjerciciosVector {
public static void main(String[] args) {
Estado estado1;
// INSTANCIACION Vector
Vector<Estado> estados = new Vector();
// INSTANCIACION DE OBJETOS Y AGLUTINADO AL Vector
estado1 = new Estado(1, "España", 46570000);
estados.addElement(estado1);
estado1 = new Estado(2, "Portugal", 10310000);
estados.addElement(estado1);
estado1 = new Estado(3, "Alemania", 82790000);
estados.addElement(estado1);
estado1 = new Estado(4, "Francia", 67120000);
estados.addElement(estado1);
estado1 = new Estado(5, "Italia", 60590000);
estados.addElement(estado1);
System.out.println("------------------ RECORRIDO size() get() ------------------");
Estado estado2;
for (int i = 0; i<estados.size(); i++)
{
estado2 = estados.elementAt(i);
// estado2 = (Estado)estados.get(i);
System.out.println("identificador de estado : "+estado2.getIdEstado());
System.out.println("nombre del estado : "+estado2.getNombre());
System.out.println("número de habitantes del estado : "+estado2.getNumeroHabitantes());
System.out.println("------------------------------------------------");
}
System.out.println("------------------- RECORRIDO Iterator --------------------");
Estado estado3;
Iterator<Estado> iteratorEstados = estados.iterator();
while (iteratorEstados.hasNext()) {
estado3 = iteratorEstados.next();
System.out.println("identificador de estado : "+estado3.getIdEstado());
System.out.println("nombre del estado : "+estado3.getNombre());
System.out.println("número de habitantes del estado : "+estado3.getNumeroHabitantes());
System.out.println("------------------------------------------------");
}
System.out.println("------------------ RECORRIDO Enumeration -------------------");
```

```
Estado estado4;
Enumeration<Estado> enumerationEstados = estados.elements();
while (enumerationEstados.hasMoreElements()) {
estado4 = enumerationEstados.nextElement();
System.out.println("identificador de estado : "+estado4.getIdEstado());
System.out.println("nombre del estado : "+estado4.getNombre()); System.out.println("número de
habitantes del estado : "+estado4.getNumeroHabitantes());
System.out.println("-----------------------------------------------");
}
}
}

package ejerciciosset;
import java.util.Collections;
import java.util.Enumeration;
import java.util.Iterator;
import java.util.LinkedHashSet;
public class EjerciciosSet {
public static void main(String[] args) {
Estado estado1;
// INSTANCIACION Set
LinkedHashSet<Estado> estados = new LinkedHashSet();
// INSTANCIACION DE OBJETOS Y AGLUTINADO AL Set
estado1 = new Estado(1, "España", 46570000);
estados.add(estado1);
estado1 = new Estado(2, "Portugal", 10310000);
estados.add(estado1);
estado1 = new Estado(3, "Alemania", 82790000);
estados.add(estado1);
estado1 = new Estado(4, "Francia", 67120000);
estados.add(estado1);
estado1 = new Estado(5, "Italia", 60590000);
estados.add(estado1);
Estado estadoADuplicar = new Estado(4, "Francia", 67120000);
estados.add(estadoADuplicar);
if (estados.contains(estadoADuplicar))
System.out.println("Ya existe este objeto en el Set");
System.out.println("------------------ RECORRIDO Iterator ------------------");
Estado estado2;
Iterator<Estado> iteratorEstados = estados.iterator();
while (iteratorEstados.hasNext()) {
estado2 = iteratorEstados.next();
System.out.println("identificador de estado : "+estado2.getIdEstado());
System.out.println("nombre del estado : "+estado2.getNombre());
System.out.println("número de habitantes del estado : "+estado2.getNumeroHabitantes());
System.out.println("-----------------------------------------------");
}
System.out.println("------------------ RECORRIDO Enumeration -------------------");
```

```
Estado estado3;
Enumeration<Estado> enumerationEstados = Collections.enumeration(estados);
while (enumerationEstados.hasMoreElements()) {
estado3 = enumerationEstados.nextElement();
System.out.println("identificador de estado : "+estado3.getIdEstado());
System.out.println("nombre del estado : "+estado3.getNombre());
System.out.println("número de habitantes del estado : "+estado3.getNumeroHabitantes());
System.out.println("-----------------------------------------");
}
}
}
```

No detallamos para dichos ejemplos la visualización en la consola de salida correspondiente a la ejecución de los mismos, porque contribuiría innecesariamente a la extensión del libro. Analicemos con detalle los mecanismos aplicados a la gestión de las colecciones gestionadas en los ejemplos.

Añadir objetos a la colección

En la clase Vector, utilizamos el método:

```
estados.addElement(estado1);
```

y en las clases *ArrayList*, y *Set*, el método:

```
estados.add(estado1);
```

Recorrido basado en el número de elementos

Utilizamos, para las clases *ArrayList*, y *Vector*, el método *size(),* que nos devuelve dicho número de elementos en la colección:

```
for (int i = 0; i<estados.size(); i++)
```

En cada repetición, se aplica el valor de la variable de control al método *get(posición)*, que nos devuelve el objeto que ocupa dicha posición en la colección. Este método nos permite un acceso directo al elemento. En el ejemplo de la clase Vector, utilizamos para ello el método *elementAt(posición)*.

Recorrido basado en la modalidad de *for* aplicable en colecciones y estructuras de datos

```
for (Estado estado3 : estados)
```

Ya tratado en su aplicación a los arrays.

Recorrido aplicando *Iterator*

Mediante el método *iterator()* de la colección, obtenemos el *Iterator* aplicable al recorrido de dicha colección:

```
Iterator<Estado> iteratorEstados = estados.iterator();
```

Como condición del recorrido en la estructura repetitiva, utilizamos

```
while (iteratorEstados.hasNext())
```

El método *hasNext()* devuelve un *boolean* con valor *true,* si queda algún elemento disponible en el recorrido de la colección, permitiéndonos avanzar en el recorrido mediante el método *next()*

```
estado3 = iteratorEstados.next();
```

que además nos devuelve el siguiente elemento disponible en dicho recorrido.

Recorrido aplicando *Enumeration*

Totalmente análogo al recorrido anterior, obteniendo el *Enumeration* en el caso de las clases *ArrayList*, y *Set*

```
Enumeration<Estado> enumerationEstados = Collections.enumeration(estados);
```

y en el caso de la clase *Vector*

```
Enumeration<Estado> enumerationEstados = estados.elements();
```

Y en cuanto a los métodos a utilizar en el recorrido, existe una total analogía con los de Iterator:

hasNext() → hasMoreElements()
next() → nextElement()

Estableciendo una comparativa entre ambas interfaces de recorrido de colecciones (*Iterator*, y *Enumeration*), es necesario matizar que *Iterator* supone una versión que avanzada. Dicha interface, dispone del método *remove()*, que permite eliminar de la colección el último elemento devuelto en el recorrido.

Eliminación de objetos de la colección

Las eliminaciones se han aplicado solamente en el ejercicio del *ArrayList*, pero son aplicables a todas las colecciones:

- **remove() .**- elimina un solo objeto de la colección
- **clear() .**- elimina todos los objetos de la colección

Obtención de un array *Object[]* con los elementos de la colección

Solamente se ha aplicado al ejercicio del *ArrayList*, pero es aplicable a todas las colecciones:

```
Object[] arrayEstados = estados.toArray();
```

En el acceso a los componentes del array obtenido, es necesario tener en cuenta que dichos componentes son referencias a *Object*, por tanto, en la invocación a métodos del objeto, es necesario aplicar cast:

```
((Estado) arrayEstados[i]).getIdEstado()
((Estado) arrayEstados[i]).getNombre()
((Estado) arrayEstados[i]).getNumeroHabitantes()
```

Observaremos que en la clase *Estado*, se han sobrescrito los métodos *equals()*, y *hashCode()*, a efectos de evitar elementos duplicados en *LinkedHashSet<Estado>*. Para dicho control de unicidad, se aplica una comprobación, automática, del identificador HashCode devuelto por el método *hashCode()* del objeto. En el caso de que coincida dicho valor con alguno ya existente, se procede a una segunda comprobación, pero con aplicando el método *equals()*. La sobrescritura del método *equals()*, ya se abordó en el tema de este libro correspondiente a Encapsulamiento. El método *hashCode()*, también presente en *Object*, y que es susceptible de que todas las clases sobrescriban, devuelve un valor *int* asociado a cada objeto, que permite el uso de hash. En nuestros ejemplos hemos utilizado la siguiente implementación de dicho método:

```
public int hashCode() {
return idEstado + nombre.hashCode() + numeroHabitantes ;
}
```

En el ejemplo EjerciciosSet, se ha intentado “forzar” un duplicado:

```
Estado estadoADuplicar = new Estado(4, “Francia”, 67120000);
estados.add(estadoADuplicar);
```

pero la existencia en la clase *Estado* de los métodos *hashCode()*, y *equals()*, hace posible que el control automático para evitar duplicidades, actúe eficazmente, e impida que se produzca dicha duplicidad.

Comprobación de si existe un objeto en la colección

Utilizamos, para ello, el método *contains()*, disponible en todas las colecciones. Se manifiesta su uso en el ejemplo que gestiona el *LinkedHashSet<Estado> estados*:

```
if (estados.contains(estadoADuplicar))
System.out.println("Ya existe este objeto en el Set");
```

En la declaración de la referencia que "apuntará" la colección a gestionar, se ha utilizado el operador "diamante"

```
ArrayList<Estado> estados = new ArrayList();
```

Dicho operador permite establecer una inferencia de tipos. Como consecuencia de su uso en nuestros ejemplos, al "rescatar" un objeto de la colección, no es necesario aplicar *cast*:

```
estado2 = estados.get(i);
```

Caso de no haberlo utilizado

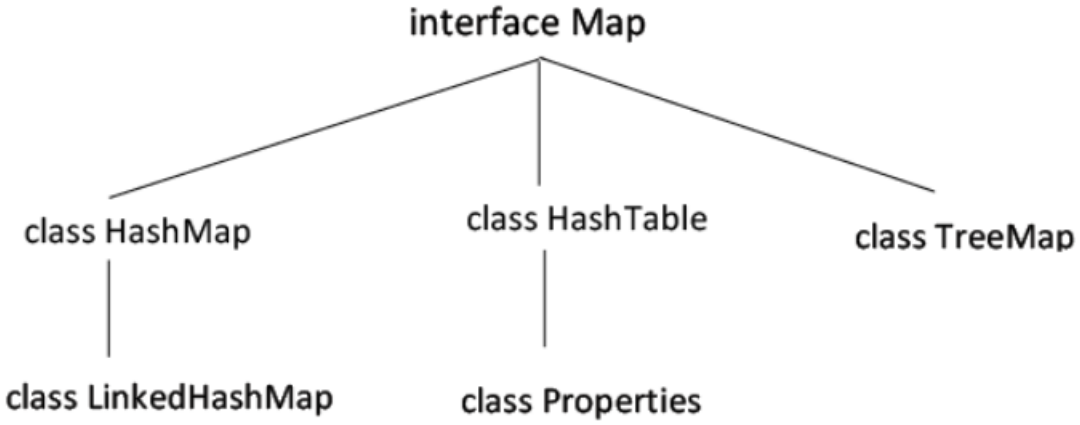
```
ArrayList estados = new ArrayList();
```

sí que hubiese sido necesario aplicar *cast* al *Object* devuelto:

```
estado2 = (Estado) listaEstados.get(i);
```

MAP

En este apartado, trataremos la jerarquía de clases correspondiente a la *interface Map*. Presentamos dicha jerarquía en el siguiente esquema:



Procedemos a continuación a una breve descripción comparativa de todas las interfaces y clases de dicha jerarquía.

- ***Map*** .- Las clases que implementan esta interface permiten almacenar un conjunto de pares:

  clave → valor

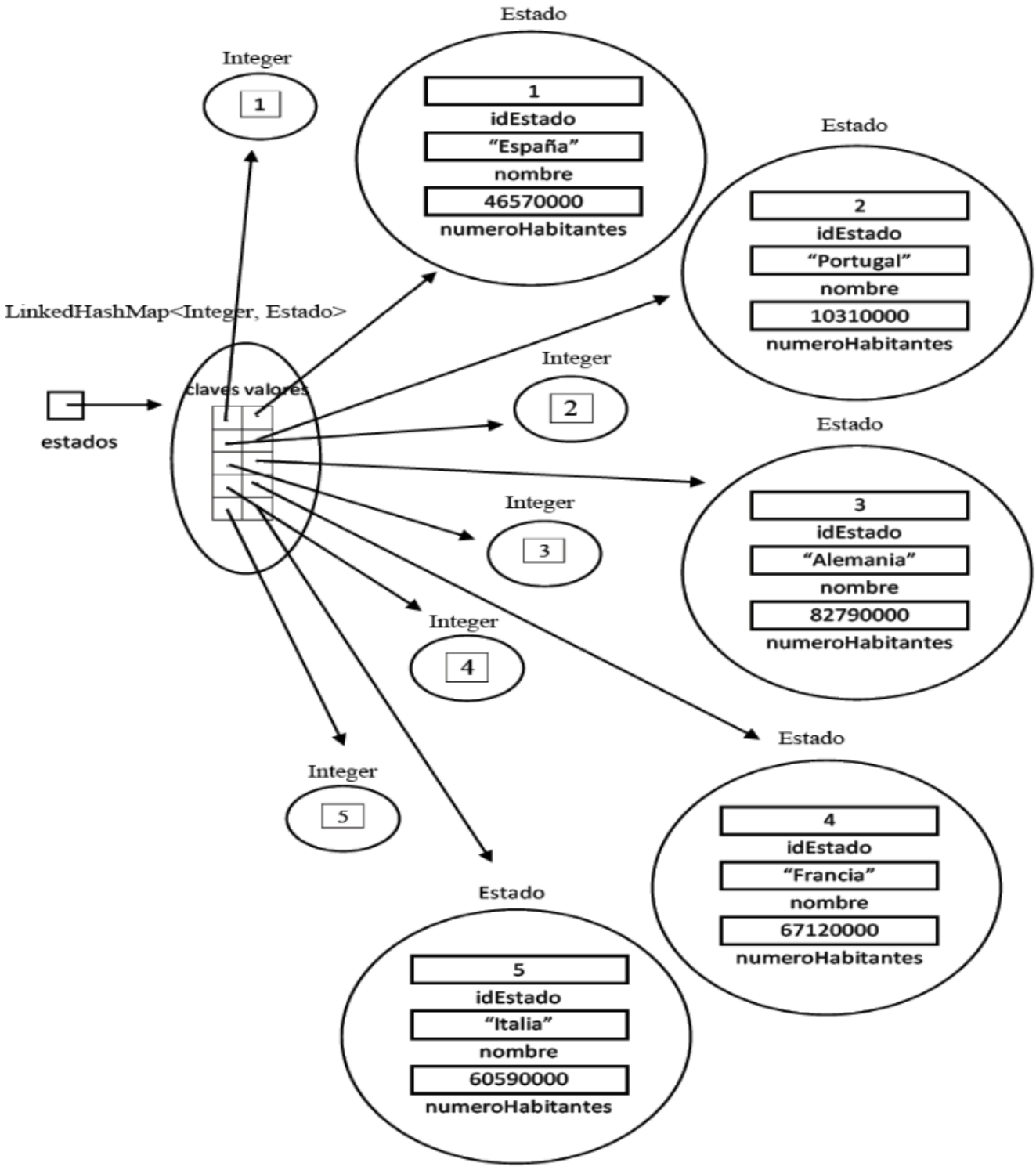
  de tal modo que un valor concreto es accesible directamente aportando el valor de clave que le corresponde.
- ***LinkedHashMap*** **.-** Los elementos quedan almacenados en el mismo orden en que se insertan.
- ***HashMap*** **.-** A diferencia de la anterior, el orden en que se recuperan los elementos, no coincide con el de inserción. Supone mayor rendimiento.
- ***TreeMap*** **.-** Los elementos se insertan según el criterio de ordenación asociado a los valores.
- ***HashTable*** **.-** Es totalmente equivalente a *HashMap*, pero enfocada a aplicaciones que requieran sincronización.
- ***Properties*** **.-** Supone una concreción de *Map*, en que tanto la clave como el valor son *String*.

Exponemos a continuación un ejemplo representativo de la gestión de dichas clases. Dicho ejemplo "aglutina" objetos de la clase **Estado**, ya expuesta y utilizada con anterioridad en este tema.

```
package ejerciciosmap;
import java.util.Iterator;
import java.util.LinkedHashMap;
```

```
public class EjerciciosMap {
public static void main(String[] args) {
Estado estado1;
// INSTANCIACION Map
LinkedHashMap<Integer, Estado> estados = new LinkedHashMap();
// INSTANCIACION DE OBJETOS Y AGLUTINADO AL Map
estado1 = new Estado(1, “España”, 46570000);
estados.put(estado1.getIdEstado(),estado1);
estado1 = new Estado(2, “Portugal”, 10310000);
estados.put(estado1.getIdEstado(),estado1);
estado1 = new Estado(3, “Alemania”, 82790000);
estados.put(estado1.getIdEstado(),estado1);
estado1 = new Estado(4, “Francia”, 67120000);
estados.put(estado1.getIdEstado(),estado1);
estado1 = new Estado(5, “Italia”, 60590000);
estados.put(estado1.getIdEstado(),estado1);
System.out.println(“------------------ RECORRIDO Iterator ------------------”);
Iterator<Integer> iteratorEstados = estados.keySet().iterator();
while (iteratorEstados.hasNext()) {
Integer keyEstado = iteratorEstados.next();
System.out.println(“identificador de estado : “+estados.get(keyEstado.intValue()).getIdEstado());
System.out.println(“nombre del estado : “+estados.get(keyEstado.intValue()).getNombre());
System.out.println(“número de habitantes del estado :
“+estados.get(keyEstado.intValue()).getNumeroHabitantes());
System.out.println(“-----------------------------------------------”);
}
System.out.println(“---- BUSQUEDA DEL VALOR CORRESPONDIENTE A UNA CLAVE ----”);
Integer claveBuscada = new Integer(2); // valor de clave existente
// Integer claveBuscada = new Integer(15); // valor de clave no existe
if (estados.containsKey(claveBuscada)) {
Estado estado2 = estados.get(claveBuscada);
System.out.println(“identificador de estado : “+estado2.getIdEstado());
System.out.println(“nombre del estado : “+estado2.getNombre());
System.out.println(“número de habitantes del estado : “+estado2.getNumeroHabitantes());
}
else {
System.out.println(“Valor de clave no existe en el Map”);;
}
}
}
```

En el siguiente esquema representamos gráficamente como se almacena la información en *LinkedHashMap<Integer, Estado>* mediante la asociación de parejas clave → valor.


Estado
Integer
1
1
idEstado
"España"
nombre
46570000
numeroHabitantes
Estado
2
idEstado
"Portugal"
nombre
10310000
numeroHabitantes
LinkedHashMap<Integer, Estado>
claves valores
estados
Integer
2
Estado
3
idEstado
"Alemania"
nombre
82790000
numeroHabitantes
Integer
3
Integer
4
Estado
4
idEstado
"Francia"
nombre
67120000
numeroHabitantes
Integer
5
Estado
5
idEstado
"Italia"
nombre
60590000
numeroHabitantes

Cada pareja clave → valor, siendo en el ejemplo:

- **clave :** *Integer*
- **valor :** *Estado*

se va añadiendo al Map mediante el método *put(clave, valor)*:

```
estado1 = new Estado(1, "España", 46570000);
estados.put(estado1.getIdEstado(),estado1);
```

Recorremos el *Map* aplicando un *Iterator* al conjunto de las claves obtenido mediante el método *keySet()*:

```
Iterator<Integer> iteratorEstados = estados.keySet().iterator();
```

Y en cada repetición recuperamos cada clave

```
Integer keyEstado = iteratorEstados.next();
```

y accedemos al valor correspondiente a dicha clave aplicando dicha clave al método get(clave):

```
estados.get(keyEstado.intValue())
```

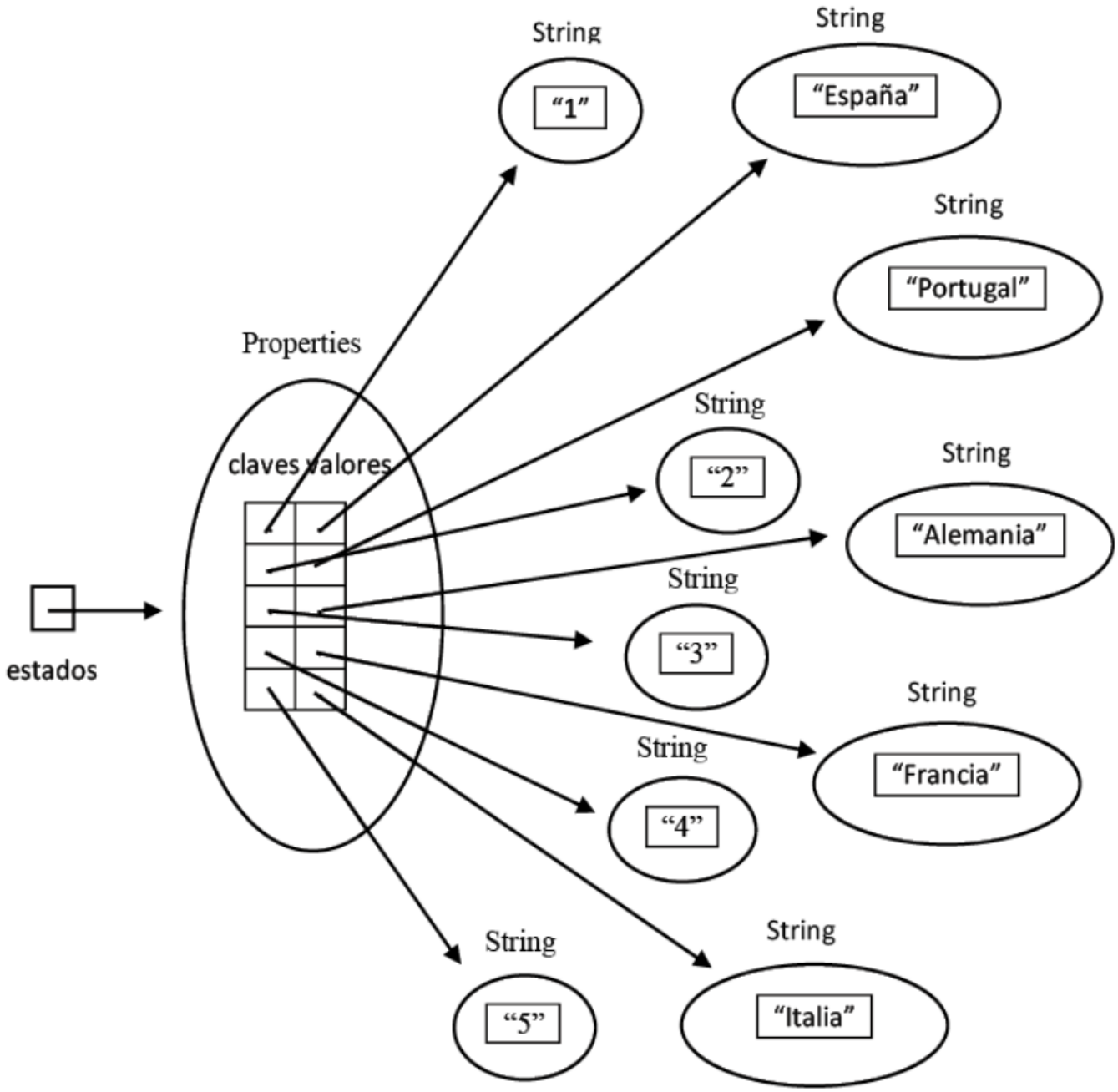
El método *containsKey(claveBuscada)* es equivalente al método *contains() de las colecciones*, pero transfiriendo como argumento el valor de la clave buscada.

Aportamos a continuación un ejemplo de gestión de *Properties*:

```
package ejerciciosproperties;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.Enumeration;
import java.util.Iterator;
import java.util.Properties;
public class EjerciciosProperties {
public static void main(String[] args) {
// INSTANCIACION Properties
Properties estados = new Properties();
// AGLUTINADO DE CADENAS AL Properties
estados.put("1", "España");
estados.put("2", "Portugal");
estados.put("3", "Alemania");
estados.put("4", "Francia");
```

```
estados.put(“5”, “Italia”);
System.out.println(“------------------ RECORRIDO Iterator ------------------”);
Iterator iteratorEstados = estados.keySet().iterator();
while (iteratorEstados.hasNext()) {
String keyEstado = (String) iteratorEstados.next();
System.out.println(“identificador de estado : “+keyEstado);
System.out.println(“nombre del estado : “+estados.getProperty(keyEstado));
System.out.println(“---------------------------------------------”);
}
System.out.println(“---- BUSQUEDA DEL VALOR CORRESPONDIENTE A UNA CLAVE ----”);
String claveBuscada = “2”; // valor de clave existente
// String claveBuscada = “15”; // valor de clave no existe
if (estados.containsKey(claveBuscada)) {
System.out.println(“identificador de estado : “+claveBuscada);
System.out.println(“nombre del estado : “+estados.getProperty(claveBuscada));
}
else {
System.out.println(“Valor de clave no existe en el Properties”);;
}
try {
estados.store(new FileOutputStream(“D:\\archivos_creados_Java\\estados.properties”), “Estados”);
Properties estadosLeidosDesdeArchivo = new Properties();
estadosLeidosDesdeArchivo.load(new FileInputStream(“D:\\archivos_creados_Java\\estados.properties”));
System.out.println(“--- RECORRIDO Enumeration Properties leido desde archivo --”);
Enumeration enumerationEstados = estadosLeidosDesdeArchivo.keys();
while (enumerationEstados.hasMoreElements()) {
String keyEstado = (String) enumerationEstados.nextElement();
System.out.println(“identificador de estado : “+keyEstado);
System.out.println(“nombre del estado : “+estadosLeidosDesdeArchivo.getProperty(keyEstado));
System.out.println(“---------------------------------------------”);
}
} catch (IOException excepcion)
{ System.out.println(“Problema con fichero para almacenar el Properties”);}
}
}
```

Representamos gráficamente en el siguiente esquema como se almacena la información en *Properties* mediante la asociación de parejas clave → valor. Como ya se ha mencionado con anterioridad, en *Properties*, tanto la clave, como el valor, son *String*.



En relación al *Map*, disponemos adicionalmente en *Properties* del método:

```
getProperty(claveBuscada)
```

para obtener el valor a partir de la clave que le corresponde

y del método

```
keys()
```

para obtener un *Enumeration* con el conjunto de las claves, que nos permita recorrer el *Properties*.

El método *store()*, permite crear un fichero en el sistema de archivos, que nos permita proporcionar persistencia a la información almacenada en *Properties*:

```
estados.store(new FileOutputStream("D:\\archivos_creados_Java\\estados.properties"), "Estados");
```

El proceso inverso se efectúa mediante la invocación al método *load()*, que nos permite recuperar desde un fichero a un *Properties*, la información que en otro momento fue guardada mediante *store()*:

```
estadosLeidosDesdeArchivo.load(new FileInputStream("D:\\archivos_creados_Java\\estados.properties"));
```